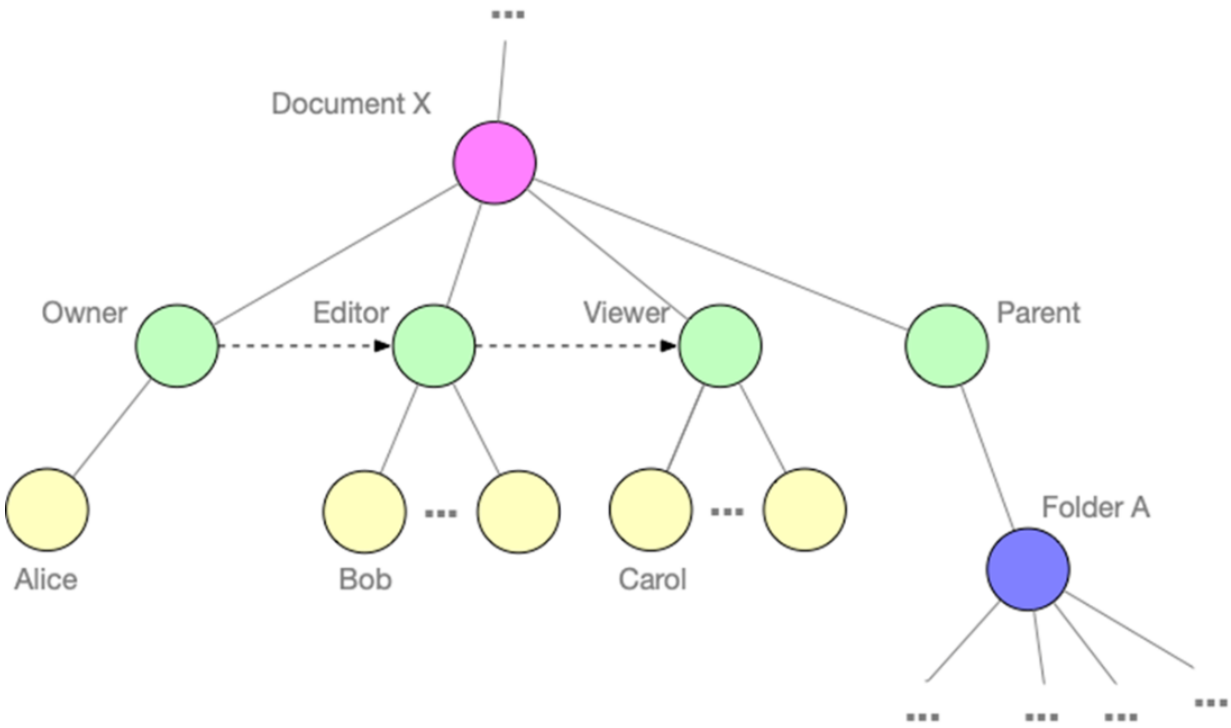



**Figure 1.3 A relationship graph representing access to a Google document. Rather than use static ACLs, this model captures roles (like Owner, Editor, Viewer) as first-class relationships between users and resources. The graph also models hierarchical relationships (such as parent folders), enabling more flexible, general-purpose authorization logic that can be queried and evaluated dynamically.**

Policy as Data is good for unstructured rules that can't be easily enumerated in a few policies. To use Google Docs again as an example, each document has a different owner, editors, and contributors. These relationships can be easily recorded in a graph database, for example, and queried at run time to make an access decision.

## 1.4.3 Dynamic authorization needs both

Policy as Code or Policy as Data is not an either-or decision for many organizations. Both have their strengths and can be used together to provide the best outcomes for a specific situation. Policy as Code platforms that can use attributes from data sources can sometimes offer the best of both worlds by writing policies that use attributes from a data store that can be updated over time.